

# 数组和链表

数组和链表是两种最基本的存储结构，他们各有自己的优缺点。

数组：

优点：可以在 $O(1)$ 的时间复杂度内找到需要操作的位置

缺点：无论是插入还是删除都需要移动后面的所有数据，所以插入和删除的时间复杂度是 $O(n)$ 。

链表：

优点：链表的插入和删除只需要更新相邻另两个结点的前驱和后继，插入和删除的时间复杂度是 $O(1)$

缺点：对于链表，我们只知道当前位置的下一个(上一个)位置，所以需要查找位置，那么我们需要遍历整个链表，直到找到位置，查找的时间复杂度是 $O(n)$ 。

# 链表

在数组中，编号为 $i$ 的元素的下一个元素一定是编号为 $i+1$ 的元素，但是在链表中，编号为 $i$ 的元素和编号为 $i+1$ 的元素并没有什么直接的联系，就想树和图一样，相连的两个结点编号并没有什么联系。

那么对于链表，我们要如何找到他的下一个元素呢，和树和图同样的道理，我们可以在当前结点存储他的数值之外，再存储他的下一个结点的编号，然后通过存储的信息找到他的下一个位置。

所以链表的查找只能从头开始查找，而且 $a[k]$ 也并不代表第 $k$ 个元素，即链表中编号和位置没有任何关系。

# 链表

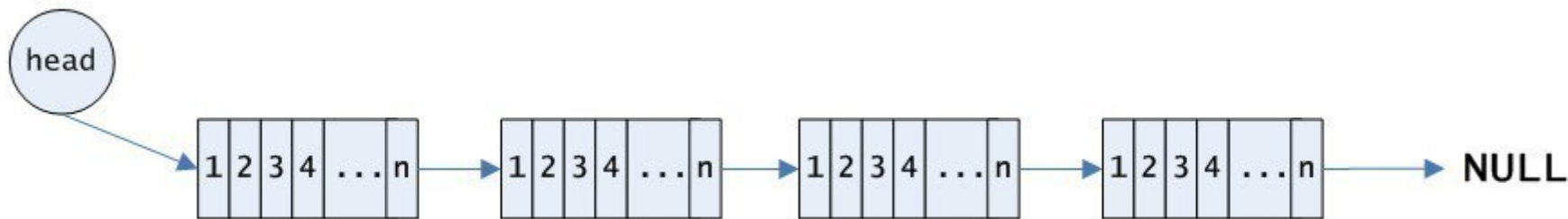
```
struct node
{
    int num;//值域
    int next;//指针域
}q[maxn];//定义一个链表中的结点
for(int i=1;i<=n;i++)
{
    q[i].num=x;
    q[i].next=i+1;//最开始时的链表和数组一样的
    //第i个元素的下一个元素就是第i+1个元素
}
q[n].next=0;//最后一个元素的下一个元素没有
int t=1;
while(t)//当t=0时，说明遍历结束了
{
    if(q[t].num==x)
        break;
    t=q[t].next;//查找下一个元素只能通过next找
}
//当我此时已经找到了插入的位置是编号为t的结点
q[k].next=q[t].next;//先更新新节点的后继
q[t].next=k;//再更新t节点的后继
//如果操作反了就会导致找不到t结点原来后继
//删除元素直接跳过该元素，更新后继就可以了
q[last].next=q[t].next;//记得记录t结点的前一个结点last
```

# 数组和链表

所以我们需要快速的删除或者插入某个指定数据，无论是用数组还是链表来实现，每次操作的时间复杂度都是 $O(n)$ 的，效率比较低，我们可以把数组和链表的优点结合起来，组成一种新的数据结构——块状链表。

我们以前学习过分块，我们可以把 $n$ 个数据分成 $\sqrt{n}$ 块，每一块大约有 $\sqrt{n}$ 个数据，我们把这 $\sqrt{n}$ 个数据看做一个整体来操作。

块状链表也是用类似的方法，同样，也是把整个数据分成 $\sqrt{n}$ 块，每一块内用数组的方式来存储，每两块之间用链表的形式来连接。



# 块状链表

块状链表的核心操作有三个：查找，插入和删除

(1) 查找需要插入的位置(查找第 $k$ 个数)

因为块状链表从宏观上看是链表，只是对于链表的每个结点他是一个顺序表，所以链表的查找只能从头开始一个一个找。

但是链表的每一个结点是一个数组，我们只需要记录立链表中每一个结点的长度就一个一个往后面找了。首先找到该结点属于哪一块，再去这一块中查找第几个值。

因为链表的节点数为 $\sqrt{n}$ 个，每一块的长度为 $\sqrt{n}$ ，所以链表查找的时间复杂度为 $\sqrt{n}$ 。

# 块状链表

## 块状链表初始化

```
const int maxn=1010;
struct node
{
    int a[maxn];
    int len;//当前每一块长度
    int next;//该块的后继的编号
}q[maxn];
int main()
{
    scanf("%d",&n);
    m=sqrt(n);
    for(int i=1;i<=n;i++)
    {
        scanf("%d",&a[i]);
        int t1=(i-1)/m+1;//属于哪一块
        q[t1].len++;//块内第几个数
        q[t1][q[t1].len]=a[i];
    }
    for(int i=1;i<n;i++)
        q[i].next=i+1;//链表
}
```

有的时候可能会第一个数前面插入一段数据，所以可以在第一块前面添加一个头指针，也就是相当于加入一个不占空间的链表起点，每次遍历从起点开始遍历。

# 块状链表

## 块状链表插入位置的查找

```
q[0].next=1; // 编号为0的块是一个虚拟的点
q[0].len=0;
int t=0;
while(q[t].next)
{
    if(k<=q[t].len) // 说明要查找的数在第t块中
        break;
    k-=q[t].len;
    t=q[t].next; // 注意是链表，不能是t++
} // 最终找到插入的位置是q[t].a[k]这个位置
```

# 块状链表

## (2) 在第 $k$ 个位置之后插入一个数 $x$

因为插入的位置在块状链表中，也就是是在数组中，所以我们可以用数组的形式进行插入，即插入点之后的位置的数都想后挪一个位置。

也有另一种插入的办法，我们找到插入位置之后，把该块分成两块，两块之间用链表的形式连接（相当于新建一个块），在第一块的末尾插入该数。

两种方法原理时间复杂度都差不多，这里主要介绍第一种做法。就是以前的数组暴力插入，因为每一块内的大小在 $\sqrt{n}$ 左右，所以每次最坏时间复杂度为 $\sqrt{n}$ 。



# 块状链表

块状链表插入一个数x

```
// 前面找到插入的位置为q[t].a[k]  
for(int i=q[t].len;i>=k+1;i--)  
q[t].a[i+1]=q[t].a[i];// 该位置之后的数后移  
q[t].a[k+1]=x;// 插入该数  
q[t].len++;// 长度+1
```

# 块状链表

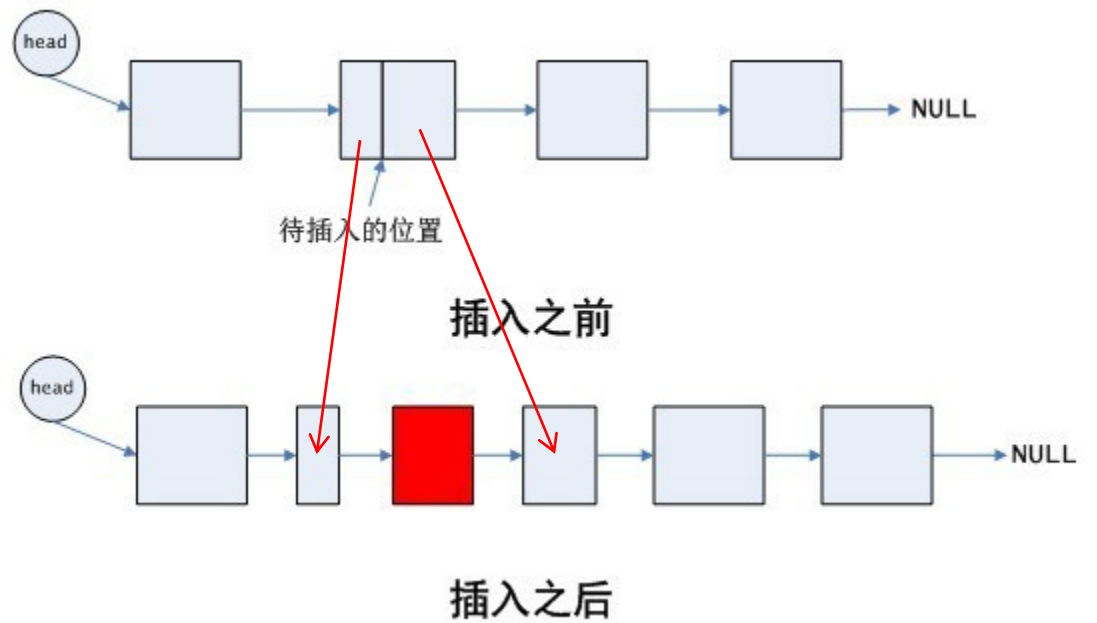
## (3) 在第 $k$ 个位置之后插入一段数据

有的时候插入的是一段数据，特别是只限制了所有操作的插入数据的总数量的时候，前面的做法就有很大的问题，因为直接插入进去之后，一个节点的空间是不够用的，但是又不能对每一个节点都开很大空间。

对于这种情况，那么我们就需要换一种插入方式。首先，我先把插入的一段数据先变成一段独立的块状链表，然后再把该块状链表插入进原来的块状链表中。

但是这种插入方式就只能用前面说的另一种插入的办法，找到插入位置后，按照插入位置把该块分成两块，然后在这两块之间插入一个块状链表。更新前驱和后继。

# 块状链表



# 块状链表

```
cnt++; // 已经分配的块的数量
for(int i=k+1; i<=q[t].len; i++)
{
    q[cnt].len++;
    q[cnt].a[q[cnt].len]=q[t].a[i];
}
q[t].len=k; // 更新原来的t这一块的信息
// 链表中插入节点的时候，一定是先更新后继，再更新前驱
q[cnt].next=q[t].next; // 更新cnt的后继为以前t的后继
q[t].next=cnt; // 更新现在t的后继为cnt

// 把新加入的一段数据按照sqrt(n)先分成块
cnt++; int l=cnt; // 记录初始块编号
for(int i=1; i<=m; i++)
{
    q[cnt].len++;
    q[cnt].a[q[cnt].len]=b[i];
    if(q[cnt].len==sqrt(n))
    {
        q[cnt].next=cnt+1;
        cnt++;
    }
}
q[cnt].next=q[t].next;
q[t].next=l;
```

# 块状链表

## (4) 删除第 $k$ 个元素

删除和插入的操作基本是一致的，先找到删除的元素，以该位置为分界线把该块分成两块，该位置是前一块的最后一个位置，相当于在数组末尾删除，很简单。

## (5) 删除区间 $[l, r]$

区间删除，因为可能会涉及到几个块，并且还会涉及到不完整的块，所以我们可以都当做完整的块删除，所以我们可以找到 $l$ 的位置，分成两块，再找到 $r$ 的位置，又分成两块，这样区间 $[l, r]$ 之间的块都是完整的，直接更新后继就可以了。

# 块状链表

```
//l的数是q[t1].a[k1]  
//r的数是q[t2].a[k2]  
//先按照前面的做法把t1,t2这两块分成两个块  
q[t1].next=q[t2].next;
```

# 块状链表

块状链表的插入删除操作相对都比较简单，但是有一个严重的问题是分块只有在块的大小和数量接近 $\sqrt{n}$ 的时候效率才比较高，但是这里涉及到多次插入删除操作，就会导致块的数量和大小发生变化，比如一直在一个位置插入一个数，这样这个块就会特别大，每次插入的时间复杂度就会特别慢，所以如果不对块的大小和数量进行修改，就会导致块状链表的时间复杂度特别高。所以我们需要对块的分裂和合并。

如果我们用的是每次找到位置都分裂的方法来做，那么就只需要学会如何对块进行合并就可以了，因为这种方式，块的大小肯定都是小于 $\sqrt{n}$ 的。

# 块状链表

块状链表当块的大小和数量都在 $m = \sqrt{n}$ 的时候效率是最高的，我们要保证每一块的大小和数量都在一定范围内，我们一般维持每一个块的大小在 $[m/2, 2*m]$ 之间，这样块的数量也在 $[m/2, 2*m]$ 之间。

因为我们这种处理方式块的大小只会变小，不会变大，所以我们维护的时候可以只需要维护：任意相邻两块的大小加起来大于 $m$ 就可以了，这样块的数量就会保证在 $[m/2, 2*m]$ 之间。

即如果相邻两块的大小加起来小于 $\sqrt{n}$ ，就把这两块合并，如果合并之后和后面的块大小纸盒还是小于，那么继续合并。这样效率就会高一些。

我们在查找的时候就可以边查找边合并，因为我们保证了块的数量，所以不用担心时间复杂度。



# 块状链表

```
int t=0;int sum=0;int last=0;
void merge(int x,int y)
{
    for(int i=1;i<=q[y].len;i++)
        q[x].a[q[x].len+i]=q[y].a[i];//赋值
    q[x].len+=q[y].len;//更新长度
    q[x].next=q[y].next;//合并之后就没有y节点了
}
while(q[t].next)
{
    if(k<=len)
        break;
    //处理到查找的位置之前的完整的块
    sum+=q[t].len;
    if(t!=0&&t!=1)//第0个块不和其他块合并
    {
        if(sum<=sqrt(n))
            merge(last,t);
        else
            sum=q[t].len;//如果不需要合并, 那么sum的值会改变
    }
    k-=q[t].len;
    last=t;//记录一下上一个节点的编号
    //因为我们这里只是记录了next, 所以只能通过前一个点找后一个点
    //没有办法通过后一个点找前一个点
    //当然我们也可以开双向链表(next,last)
    //但是双向链表更新前驱后继相对比较麻烦, 容易忘记更新
    t=q[t].next;
}
```



# 练习题

维护数列

文本编辑器

弹飞绵羊

